ChessDecide: Benchmarking Decision Models at Chess

As of 2026-11-01. The PDF edition of this report, `docs/BENCHMARK-REPORT.pdf`, is built from this file with `node scripts/build-report-pdf.mjs`.

Authorship. This report was written by Claude Sonnet 5.5 using extensive data from the benchmark runs themselves. The code behind it was written by two AI coding agents: Claude Sonnet 5.5 (via Claude Code) and DeepSeek V4.1 Flash 1017 (via the Pi coding harness). The benchmark sweep ran in a Claude Code on the web session over about 30 hours.

Abstract

Decision models, which pick one option from a menu and return no reasoning, play chess at 122 to 711 Elo when the menu is bare and 249 to 886 Elo when each option carries a few rule-derived facts. We measured 20 catalog models this way, 50 games per model per arm, against a calibrated ladder of engine opponents.

- **Scale of the run:** 2,002 finished games, 142,635 plies and 70,806 successful model calls, for $2.57 of API spend. Every model move was also judged by Stockfish.
- **Hints are the largest effect:** the `hinted` arm lifts all 20 models, by +56 to +465 Elo (median about +290). Nothing in a hint is an engine evaluation: captures, castling, promotion, check and mate, all derivable from the rules.
- **The ranking is tiers, not an order:** with 50 games the 95% interval is about ±105 Elo, so most neighbouring ranks are statistically tied.
- **Interface is a capability:** 5 of the 25 catalog models could not play at all, because they cap a question at 20 or 26 options or reject a structured state. That is a result about the models, not a harness bug.

Everything below is meant to be reproducible: the protocol is byte-specified, the opponents are calibrated and versioned, and the raw series and publish SQL are in the repository.

Motivation

We wanted to know how much of a decision model's weakness at a rule-bound task is the model, and how much is the way the question is asked.

OpenRouter's Decisions endpoint (`POST /api/alpha/decisions`) takes a structured state and a multiple-choice question and returns the model's pick plus a probability for every option. There is no text output and no chain of thought, so the only levers an evaluator has are what the model sees and how the options are worded.

Chess suits this for three reasons:

1. **The answer space is finite and legal.** Every position has a complete list of legal moves, so the menu is exact and nothing can be hallucinated outside it.
2. **Quality has an external judge.** Stockfish scores every legal move, so each pick has a measurable cost in centipawns and win probability.
3. **Strength has a scale.** A ladder of engines of known strength turns a pile of game results into one number, Elo, with an uncertainty.

The central comparison is two arms that differ in one thing. The `clean` arm describes a move as `Queen h5 to f7`. The `hinted` arm describes it as `Queen h5 to f7, captures pawn, checkmate`, and adds an `in_check` flag to the state. Same models, same opponents, same scoring; only the wording of the options changes.

Environment

The whole sweep ran headless from one command-line runner on a single cloud Linux VM (4 cores, about 15 GB RAM, Node 22, Stockfish 16 from the distribution package). The core game logic is pure TypeScript with no I/O; engines, the model client, the store and the scheduler sit behind interfaces, so the same code can run locally or on Cloudflare.

System overview. The runner reads a plan (models, arms, caps) and plays games one ply at a time, two games per model at once and up to four models at once, in resumable 110-minute slices. For each move it calls three things: the Decisions API (a pick and a probability per option), a Stockfish opponent (ladder levels, 50 ms per move) and a Stockfish judge (every legal move, depth 14). Everything it learns is written to a local SQLite store. The report bundle (report, JSON, SQL) is generated from the store alone. Publishing to production D1 is a separate step on a machine with the Cloudflare login.

The runner is the only process that calls the model API or the engines. Everything it learns lands in the store, and the report bundle is generated from the store alone, which is why a broken report can be rebuilt without replaying a game.

- **Model client:** every request goes to the Decisions endpoint through one budgeted client with retries, a rate limiter and a cache. Each attempt is stored with its request, response, latency, token counts and cost.
- **Opponents and judge:** both are Stockfish processes behind the same engine interface. Ladder opponents run at `Threads=1`, `Hash=32`, 50 ms per move. The judge runs at `Threads=1`, `Hash=64`, depth 14 on every ply of every game.
- **Store:** a local SQLite database through wrangler's D1 emulation (`games`, `plies`, `attempts`, `series`, `ladder_levels`). It reached 295 MB.
- **Output:** after each cell the runner writes a bundle (report, per-series JSON export, publish SQL) and commits it, so an interrupted run loses at most the cell in flight. Publishing to the production database is a separate step on a machine with the Cloudflare login.

The sweep was driven in time-sliced runs of 110 minutes, up to four models at once and two games at a time per model, and took about 30 hours of wall clock including restarts of the VM. Cost and operations are covered at the end.

Protocol

Each model move is one request: a state, and one question with one option per legal move. The request builders are pure functions pinned by byte-exact golden tests (`PROMPT_VERSION = 3`), so the bytes sent in the first game are the bytes sent in the last.

```
{
  "model": "typesafe/jev-1.14",
  "state": {
    "game": "chess",
    "fen": "r1bqkb1r/pppp1ppp/2n2n2/4p2Q/2B1P3/8/PPPP1PPP/RNB1K1NR w KQkq - 4 4",
    "side_to_move": "white",
    "move_number": 4,
    "history_san": ["e4", "e5", "Qh5", "Nc6", "Bc4", "Nf6"]
  },
  "questions": { "move": {
    "type": "choice",
    "instructions": "Choose the strongest legal chess move for White in this position.",
    "criteria": { "c4b5": "Bishop c4 to b5", "h5f7": "Queen h5 to f7", "h5h4": "Queen h5 to
        h4" }
  } }
}
```

(Options trimmed to three; real requests list every legal move, sorted by UCI string.) The model returns its pick and a probability for each option. No session id is sent, and nothing about the opponent, the game result or the ladder.

The two arms

	`clean`	`hinted`
Option key	UCI, e.g. `h5f7`	SAN, e.g. `Qxf7#`
Option text	`Queen h5 to f7`	`Queen h5 to f7, captures pawn, checkmate`
Annotations	none	captures, castling, promotion, then `check` or `checkmate`
State	`fen`, side, move number, last 16 plies	the same, plus `in_check`

The `clean` arm is deliberately bare: a capture, a quiet move and an en passant capture look identical in form, and the model must work out from the position what each move does. Note that `hinted` changes three things together, namely the key format, the descriptions and the `in_check` flag. SAN keys already carry `x`, `+` and `#`, so part of the lift may come from the keys alone. We did not run the ablations that would separate them.

An `image` arm (board picture plus text) exists in the code and was not used in this sweep.

Handling answers

- **Validation:** an unknown choice is retried once, then fails; probabilities are renormalised for metrics, with the raw sum stored.
- **Retries:** HTTP 429, 5xx and network errors get up to 3 attempts with exponential backoff and jitter. Each HTTP call is its own stored attempt.
- **Forced moves:** with one legal move the game plays it without calling the model. Those plies are excluded from every model metric.
- **Aborts:** a game whose move fails after retries is `aborted` and excluded, never scored as a loss. 28 of 2,030 games aborted in this sweep.

Opponents and the ladder

A model's Elo is where it sits on a ladder of 12 deterministic opponents whose strengths were measured by playing them against each other. Every number in this report is on ladder version 2, calibrated on 2026-10-28. Ratings from different ladder versions are not comparable.

- `random`**:** a uniformly random legal move.
- `eps75`**,** `eps50`**,** `eps30`**,** `eps15`**:** with probability ε play a uniformly random legal move, otherwise play as `uci1320`.
- `uci1320` **to** `uci2800`**:** Stockfish with `UCI_LimitStrength` and `UCI_Elo = N`, 50 ms per move, one thread.

Calibration is engine against engine, CPU only: games between each level and its neighbours on fixed seeded 4-ply openings with colours swapped, then one Elo per level fitted by maximum likelihood over all games, with a single constraint: `uci1320` is 1320. The standard errors below come from that fit.

Level	Elo	± SE	Calibration games	Faced in this sweep
`random`	160	25.7	400	139
`eps75`	391	19.2	600	436
`eps50`	614	17.6	800	629
`eps30`	819	17.8	800	566
`eps15`	1037	18.9	800	30
`uci1320`	1320	0 (anchor)	800	202
`uci1500`	1583	17.0	800	0
`uci1700`	1744	15.4	800	0
`uci1900`	1879	15.2	800	0
`uci2100`	2016	16.7	800	0

Level	Elo	± SE	Calibration games	Faced in this sweep
`uci2400`	2250	21.3	600	0
`uci2800`	2669	39.3	400	0

Two things follow from the last column. First, the opponent for each block of five games is chosen adaptively: the level nearest to the model's current estimated Elo, starting from the level nearest 1200. That keeps scores near 50%, which is where an Elo estimate is most precise, so weak models mostly faced `eps75`, `eps50` and `eps30`. Second, no model ever reached the Stockfish levels above `uci1320`, so every rating here rests on the part of the scale built from random-move mixtures plus one engine anchor. Ratings are comparable between models on this ruler; they are not human ratings.

Scoring

The headline number is a Bayesian Elo estimate with a 95% credible interval, and it is always shown next to the move-quality numbers that explain it. A rating alone is never reported.

Game result

The model scores 1, ½ or 0 per game. Draws (stalemate, insufficient material, threefold or fifty-move, and the 200-ply cap) count ½. Aborted games are excluded, never counted as losses. To save judge time, a game is adjudicated for the side ahead if, from ply 30, the depth-14 evaluation stays at or beyond ±900 centipawns for 6 plies in a row. Adjudicated games are flagged.

Elo posterior

For each series we compute a posterior over the model's Elo on a grid from 0 to 3500. A weak normal prior (mean 1200, standard deviation 600) is combined with the likelihood of every game against its opponent's calibrated Elo `R_g`, with a draw treated as half a Bernoulli outcome:

```
e_g(R) = \frac{1}{1 + 10^{(R_g - R)/400}}
\log \mathrm{post}(R) = -\tfrac12\left(\frac{R-1200}{600}\right)^2 + \sum_g \left[ s_g \ln
        e_g(R) + (1-s_g)\ln\bigl(1-e_g(R)\bigr) \right]
```

The reported Elo is the posterior mean, and the interval is the 2.5% and 97.5% posterior quantiles. The quasi-likelihood needs no draw model and is slightly conservative when draws are frequent. At 50 games and a score near 50% the half-width is about 96 Elo in theory (`1.96 · 695 · √(0.25/n)`); the observed half-widths were 102 to 112.

Comparing two series

Two series are called separated only if the 95% interval of their Elo difference excludes zero. We compute that difference by convolving the two grid posteriors, and also report P(A stronger than B). This is less conservative than requiring the two intervals not to overlap: intervals that overlap slightly

can still give a difference interval that excludes zero. So a "separated" verdict is a statement about that one pair, not a licence to rank every neighbour.

Move quality

Independently of the result, Stockfish scores every legal move at every model ply by the same procedure (depth 14), and the pick is measured against the best move.

Quantity	Definition
`wp`	mover's win probability in percentage points, a draw counting half
`wl`	`wp(best) − wp(pick)`, win probability lost by the pick
acceptable	`wl ≤ 2` (the `Acc.` column in the reports)
blunder	`wl ≥ 20`
ACPL	mean centipawn loss over the model's plies, cp clamped to ±1000

These are reported per series with the Elo, so a rating can be read against how the model actually played. Forced moves are excluded.

Design and variables

The unit of measurement is a series: one model on one arm, 50 finished games, one repeat. A plan expands `models × arms` into series (cells) and runs them with caps on spend.

What varied

- **Model:** 25 catalog models were attempted; 20 could play.
- **Arm:** `clean` or `hinted`.
- **Opponent:** chosen adaptively per block of five games, so it differs between models by design. It is the instrument, not a treatment.

What was held fixed

Variable	Value
Ladder	version 2, calibrated 2026-10-28
Prompt builders	`PROMPT_VERSION = 3`, byte-exact goldens
Question wording	`neutral` template
State	`slim`, last 16 plies of SAN history
Option order	sorted by UCI string
Game length	50 finished games, 200-ply cap
Judge	Stockfish, depth 14, one thread, 64 MB

Variable	Value
Opponent move time	50 ms per move
Openings	seeded random openings, each used for a pair of games with colours swapped
Repeats	1 (no repeated positions or resampling)

Holding these fixed also means they were not measured. We did not vary option order, history length, wording, or key format separately from the descriptions, and the two arms played different games rather than the same positions.

What was recorded

Every attempt stores the model slug, the model and provider that answered, the full request and response (compressed), latency, input and output tokens, cost, the option order and the seed. Every ply stores the move, the position after it and its judge numbers. That is why the series and publish SQL can be rebuilt from the database alone.

Plans actually run

Plan	Models	Finished cells	Finished games	Spend
`joey-1-v1`	1	2	100	$0.03
`spread-v1`	5	10	500	$0.40
`rest-clean-v1`	13 (10 played)	10	500	$0.44
`rest-hinted-v2`	10	10	500	$0.28
`premium-v1`	3	6	300	$1.30
`tail-v1`	3 (1 played)	2	100	$0.12

Two more games were played by models that then failed (one each for `solar-decide` and `tev1-4b`), making 2,002 finished games in all.

Guardrails

Each plan sets a cap per series and per run, and a pre-flight check prints a cost estimate and verifies the API key, Stockfish, the ladder, the database and a git identity before anything is spent. A series stops on its budget cap, or after three consecutive aborted games (`stopped_error`). Runs resume where they stopped without re-spending, and cells of the same model never run at the same time, to respect per-model rate limits.

Results

Elo on ladder v2: the value is the posterior mean, the bracket the 95% interval, 50 games per value. Sorted by `hinted` Elo. The "gain" column is the `hinted` mean minus the `clean` mean, both rounded. (For scale, the ladder levels are `random` 160, `eps75` 391, `eps50` 614 and `eps30` 819. The live leaderboard draws this table as a chart.)

#	Model	clean	hinted	Gain
1	baseten/mammoth-1	635 (532–740)	886 (782–990)	+251
2	cloudflare/clef	711 (607–818)	840 (736–946)	+129
3	nace-ai/drex-max	452 (345–558)	829 (726–932)	+377
4	baseten/elephant-1	496 (386–601)	819 (714–923)	+323
5	cloudflare/clef-omni	385 (278–490)	813 (710–916)	+428
6	nace-ai/drex-v2.0	621 (516–727)	794 (689–898)	+173
7	perplexity/pplx-decider-v1.2	494 (387–602)	775 (671–879)	+281
8	typesafe/jev-1.13	346 (239–450)	775 (671–879)	+429
9	liquid/d1-ultra	711 (604–816)	767 (663–871)	+56
10	perplexity/pplx-decider-flash-v1.2	302 (192–410)	767 (663–871)	+465
11	liquid/d1	563 (457–668)	752 (645–857)	+189
12	openai/gpt-6-luna-decisions	277 (166–388)	741 (635–845)	+464
13	inception/mercury-decide	297 (186–410)	732 (623–840)	+435
14	typesafe/jev-1.14	503 (396–611)	723 (617–829)	+220
15	baseten/joey-1	379 (274–483)	721 (613–830)	+342
16	nace-ai/drex-v1.5	411 (307–515)	718 (613–825)	+307
17	liquid/d1.1	503 (396–610)	716 (608–826)	+213
18	microsoft/microsoft-decision-1	494 (388–602)	709 (605–816)	+215
19	respan/span-02	311 (201–418)	641 (534–746)	+330
20	jaredpalmer/kev-4b	122 (23–227)	249 (141–359)	+127

Every model improves with hints, and the improvement is larger than the gap between most models. The gain runs from +56 (`liquid/d1-ultra`) to +465 (`perplexity/pplx-decider-flash-v1.2`), with a median of about +290, and the weakest `clean` models gain the most.

- **Move quality tells the same story.** Across the 20 complete series, the median ACPL falls from 249 to 129, the acceptable-move rate rises from 70% to 82%, and the blunder rate falls from 23% to 12%. Games also run shorter (median 77 plies to 61), and median cost per game falls from $0.0009 to $0.0005, with shorter prompts (median 630 to 506 input tokens per call) and shorter games both contributing.

- **The top is clear, the middle is a tie.** `mammoth-1` is highest on `hinted` (886) and `clef` and `d1-ultra` are highest on `clean` (711 each). Ranks 3 to 18 on `hinted` span 709 to 829, inside the roughly ±105 interval of each, so we make no claim about their order.
- **The bottom is clear too.** `kev-4b` is lowest on both arms (122 and 249); in play it scored about 50% against the random mover and 0% against `eps50` and `uci1320`.
- **No model reached the Stockfish levels.** Every game was played against `random`, the `eps` mixtures or `uci1320`, so the ratings describe where each model sits among those, not how it would fare against a strong engine.

Limitations

The comparisons between models on this ruler are reliable to the stated intervals. The absolute numbers, and several design choices, are weaker than they look.

- **Stockfish version mismatch.** Ladder v2 was calibrated with Stockfish 19 (recorded in the ladder file). The sweep ran on a host whose package provides Stockfish 16, and the run reports record that: opponents and judge were version 16. The `uci1320` anchor, the `eps` mixtures built on it, and every judge number could therefore sit slightly off the calibrated scale. We have not measured the size of the shift.
- **The scale is extrapolated downward.** Every rating here sits below the one engine anchor, in territory defined by random-move mixtures. A model that avoids blunders against random play can lose to a real engine, so these are placements on this ruler, not human-comparable ratings.
- **Small samples, one repeat.** 50 games per cell gives about ±105 Elo. There is one series per model and arm, so run-to-run variance is not measured beyond that interval, and the ladder levels' own uncertainty (15 to 40 Elo) is not propagated into it.
- **Two interfaces changed at once.** `hinted` swaps UCI keys for SAN keys, adds move annotations and adds `in_check`. SAN keys already encode captures and checks, so the effect of the annotations alone is unknown. The two arms also played different games.
- **Fixed option order.** Options are sorted by UCI string and never shuffled. Position or order bias in a model is therefore built into every result, and could differ between the arms.
- **Selection among models.** Five of 25 catalog models could not play: `clef-flash`, `solar-decide` and `solar-decide-flash` (26-option cap), `tev1-4b-experimental` (20-option cap) and `classify-1` (rejects structured state). The ranking covers only models whose interface accepts this protocol. A single `probe` call did not predict failure: `clef-flash` accepted a 48-option probe and still failed in real games.
- **Excluded games.** 28 of 2,030 games aborted and were dropped, 2 of them `gpt-6-luna-decisions` refusals at single positions. If aborts cluster on hard positions, ratings are slightly flattered.

- **Adjudication and judge depth.** Games ended early under the ±900 cp rule count as wins for the side ahead, and ACPL depends on the depth-14, single-thread judge, so it is comparable within this report only.
- **A snapshot.** The endpoint is an alpha API, models were addressed by canonical slug, and the sweep ran from 2026-10-30 to 2026-11-01. The catalog and the models behind it will change.

Operations and lessons

The whole sweep cost $2.57 on a $15 key: 70,806 successful calls and about 42.8 million input tokens, at a mean latency of 404 ms. The median game cost under a tenth of a cent. The run was cheap enough that the caps were guardrails, not budgets.

Reproducing it

- **Plans are the record.** Each plan JSON lists models, arms, size, ladder version, seed, caps and concurrency, and every report stores the plan's SHA-256, the Stockfish and Node versions and the git commit.
- **Runs resume.** Re-running the same command continues unfinished cells and skips finished ones without re-spending. Each finished cell is committed with its report, series export and publish SQL.
- **Everything is in the repository** (`AutumnsGrove/ChessDecide`): plans in `bench/plans/` , bundles in `docs/build/reports/bench-*` , the step-by-step runbook in `docs/BENCH-RUNBOOK.md` .

What went wrong, and what it taught us

1. **Detached processes died with the container.** The first two launches were killed within minutes of the last tool call, because the host evicts containers with no live foreground work. The fix was to run the sweep attached, in 110-minute slices that exit with a distinct code (75) when there is more to do, and to relaunch the same command.
2. **A dropped socket broke the report.** The report recomputes Elo for every cell and blocks the event loop for about 25 seconds on a large run. The local database emulator closed the idle connection meanwhile, and the next query failed with `other side closed` . Reads are now retried on a fresh connection.
3. **A leaked connection hung the process.** The report opened its own database connection and closed it only on success, so one thrown error left a child process holding the runner open. Both paths now close in a `finally` .
4. **A fresh database lacked the ladder.** Adaptive plans never seeded the ladder table, so per-cell commits failed on a clean machine until we ran a fixed-ladder plan once by accident. Preflight now seeds it.
5. **The cost estimator was 2.6 to 3.9 times too high.** It assumed 1,750 input tokens per call; the measured mean was about 600. It is recalibrated and now lands within about ±25% of actual spend.

6. **Concurrent reads stalled the writer.** Polling the database for status during a sweep coincided with short `SQLITE_BUSY` stalls of a few minutes (we did not isolate the cause), so progress was read from the run log instead.

Future work

Nothing here is scheduled. Each item needs a pilot before anyone decides to re-run the roster, and the first three are the cheapest way to make this result firmer.

1. **Separate the three changes inside `hinted`.** Run SAN keys with plain descriptions, UCI keys with annotations, and `in_check` alone, so the lift can be attributed.
2. **Repeat on the matching Stockfish.** Re-run the ladder calibration and a subset of series on one Stockfish version, to measure the shift described under Limitations.
3. **Pair the games.** Play both arms from the same openings and shuffle option order, so the arm comparison is paired and order bias is measured instead of fixed.
4. **New rule-derived arms.** Attack and defence facts, hanging pieces, net material change, a board drawing, and averaging over shuffled option orders. Hints must stay derivable from the rules; anything that encodes engine judgment turns the arm into a measurement of the engine.
5. **Image plus text.** Some of the models accept images, so a board image next to the text options is worth testing on that subset.
6. **A bracket protocol.** Split the menu into groups the model accepts and play rounds, so capped models can be measured. It changes what is measured, so it would be a separate, labelled arm, and a "maximum options accepted" column would report the limit itself.

Items 4 to 6 are written up in the repository's issue tracker (issue 3).